

Appendix: Complete R Code for FINSTAD Case 1

CRUZ, Ricardo Miguel Iñigo GALEDO, Enrique Lorenzo Hermoso
SEBALLOS, Josiah Dweyn Panganiban SEECHUNG, Camille Castro
SISON, Aaron Joshua Estacio

```
# Load necessary libraries
library(dplyr)
library(tidyr)
library(lubridate)
library(DBI)
library(RSQLite)
library(ggplot2)
library(scales)

# Set working directory to project root so data paths resolve consistently
# Ensure data paths resolve correctly regardless of working directory
if (dir.exists("data")) {
  data_dir <- "data"
} else if (dir.exists("../data")) {
  data_dir <- "../data"
} else {
  stop("Cannot find data/ directory. Run from project root or scripts/.")
}

# Global options
options(stringsAsFactors = FALSE, scipen = 999, width = 72)

# Helper to parse BDO volume strings (e.g. "2.50M" to 2500000)
parse_bdo_volume <- function(x) {
  x <- gsub(",", "", x)
  mult <- ifelse(grepl("M", x, ignore.case = TRUE), 1e6,
                 ifelse(grepl("K", x, ignore.case = TRUE), 1e3, 1))
  x_num <- as.numeric(gsub("[A-Za-z%]", "", x))
  x_num * mult
}

# Define tickers
```

```
us_asset <- "AAPL"
ph_asset <- "BDO.PS"
crypto <- "BTC-USD"
```

Sec 4: Data Cleaning – GALEDO, Enrique Lorenzo Hermoso (data loading) and SISON, Aaron Joshua Estacio (cleaning)

Data cleaning operations: standardise dates, filter 2020-2025, count missing/duplicates, calculate daily returns.

```
# Data Loading -- GALEDO, Enrique Lorenzo Hermoso (Sec 3: Data Acquisition)

# Load all 8 CSVs from the data directory
aapl <- read.csv(file.path(data_dir, "AAPL.csv"), stringsAsFactors = FALSE)
bdo  <- read.csv(file.path(data_dir, "BDO.csv"), stringsAsFactors = FALSE,
  ↪ fileEncoding = "UTF-8-BOM")
btc  <- read.csv(file.path(data_dir, "BTC.csv"), stringsAsFactors = FALSE)
spy  <- read.csv(file.path(data_dir, "SPY.csv"), stringsAsFactors = FALSE)
cpi   <- read.csv(file.path(data_dir, "CPI.csv"), stringsAsFactors =
  ↪ FALSE)
dgs10 <- read.csv(file.path(data_dir, "DGS10.csv"), stringsAsFactors =
  ↪ FALSE)
fedfunds <- read.csv(file.path(data_dir, "FEDFUNDS.csv"), stringsAsFactors =
  ↪ FALSE)
vix     <- read.csv(file.path(data_dir, "VIX.csv"), stringsAsFactors =
  ↪ FALSE)

# Record original observations before any processing
obs_original <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
  ↪ "VIX"),
  Original_Obs = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)
```

Dataset	Original Observations
AAPL	2,512
BDO	2,441
BTC	3,652
SPY	2,514
CPI	952
DGS10	16,105
FEDFUNDS	863

Dataset	Original Observations
VIX	9,216

```
# Sec 4: Data Cleaning -- SISON, Aaron Joshua Estacio

# Standardise date formats
# AAPL, BTC, SPY: POSIXct-style strings like "2016-06-30 00:00:00-04:00"
aapl <- aapl |> mutate(Date = as.Date(ymd_hms(Date)))
btc  <- btc  |> mutate(Date = as.Date(ymd_hms(Date)))
spy  <- spy  |> mutate(Date = as.Date(ymd_hms(Date)))

# BDO: MM/DD/YYYY
bdo <- bdo |> mutate(Date = mdy(Date))

# CPI, DGS10, FEDFUNDS, VIX: YYYY-MM-DD
cpi    <- cpi    |> mutate(Date = ymd(Date))
dgs10  <- dgs10  |> mutate(Date = ymd(Date))
fedfunds <- fedfunds |> mutate(Date = ymd(Date))
vix    <- vix    |> mutate(Date = ymd(Date))

# Filter ALL datasets to 2020-01-01 to 2025-12-31
start_date <- as.Date("2020-01-01")
end_date   <- as.Date("2025-12-31")

aapl <- aapl |> filter(Date >= start_date & Date <= end_date)
bdo  <- bdo |> filter(Date >= start_date & Date <= end_date)
btc  <- btc |> filter(Date >= start_date & Date <= end_date)
spy  <- spy |> filter(Date >= start_date & Date <= end_date)
cpi   <- cpi   |> filter(Date >= start_date & Date <= end_date)
dgs10 <- dgs10 |> filter(Date >= start_date & Date <= end_date)
fedfunds <- fedfunds |> filter(Date >= start_date & Date <= end_date)
vix    <- vix    |> filter(Date >= start_date & Date <= end_date)

# Observations after filtering
obs_filtered <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Filtered_Obs = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    ↪ nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)

# Missing values count
missing_summary <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
```

```

Missing_Values = c(sum(is.na(aapl)), sum(is.na(bdo)), sum(is.na(btc)),
                    sum(is.na(spy)), sum(is.na(cpi)), sum(is.na(dgs10)),
                    sum(is.na(fedfunds)), sum(is.na(vix)))
)

# Duplicate rows count
dup_summary <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX"),
  Duplicates = c(sum(duplicated(aapl)), sum(duplicated(bdo)),
    ↪ sum(duplicated(btc)),
    ↪ sum(duplicated(spy)), sum(duplicated(cpi)),
    ↪ sum(duplicated(dgs10)),
    ↪ sum(duplicated(fedfunds)), sum(duplicated(vix)))
)

# Clean BDO: parse Vol. column if it exists
if ("Vol." %in% colnames(bdo)) {
  bdo <- bdo |> mutate(`Vol.` = parse_bdo_volume(`Vol.`))
}

# Calculate daily returns for OHLC assets
# Daily_Return = (Close / lag(Close)) - 1
# For BDO the Price column is the close price
aapl <- aapl |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)
bdo <- bdo |> arrange(Date) |> mutate(Daily_Return = (Price / lag(Price))
  ↪ - 1)
btc <- btc |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)
spy <- spy |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
  ↪ - 1)

# Note: BDO's Change. column can be used to verify Daily_Return computation
# The first observation of each asset will have NA for Daily_Return (no
  ↪ prior day)

```

Dataset	After Filtering	Missing	Duplicates
AAPL	1,508	0	0
BDO	1,468	0	0
BTC	2,192	0	0
SPY	1,508	0	0
CPI	71	0	0
DGS10	1,500	0	0

Dataset	After Filtering	Missing	Duplicates
FEDFUNDS	72	0	0
VIX	1,535	0	0

FRED data (CPI, DGS10, FEDFUNDS) has fewer rows due to monthly or business-daily reporting frequency. Missing values introduced by join operations are handled in Section 5.

Sec 5: Database Integration – SISON, Aaron Joshua Estacio

Perform joins and justify strategy.

```
# Sec 5: Database Integration -- SISON, Aaron Joshua Estacio

# Prepare asset data for joining
# Standardise column names: BDO uses Price (close price) and Vol. (volume)
bdo_clean <- bdo |> rename(Close = Price, Volume = `Vol.`)

# Add Asset identifier to each OHLC dataset and select common columns
aapl_clean <- aapl |> mutate(Asset = "AAPL") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
bdo_clean <- bdo_clean |> mutate(Asset = "BDO") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
btc_clean <- btc |> mutate(Asset = "BTC") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
spy_clean <- spy |> mutate(Asset = "SPY") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)

# Combine all assets into one long-format table
assets_all <- bind_rows(aapl_clean, bdo_clean, btc_clean, spy_clean)

# Pivot to wide format: one row per date, columns per asset
assets_wide <- assets_all |>
  select(Date, Asset, Close, Daily_Return) |>
  pivot_wider(names_from = Asset,
              values_from = c(Close, Daily_Return),
              names_sep = "_")

# Prepare macro datasets with descriptive column names
cpi_clean <- cpi |> rename(CPI = Value)
dgs10_clean <- dgs10 |> rename(DGS10 = Value)
fedfunds_clean <- fedfunds |> rename(FEDFUNDS = Value)
vix_clean <- vix |> rename(VIX = Value)
```

```

# Demonstration of all four join types

# 1) left_join: keep all asset dates, attach macro data where available
left_example <- assets_wide |>
  left_join(cpi_clean, by = "Date") |>
  left_join(dgs10_clean, by = "Date") |>
  left_join(fedfunds_clean, by = "Date") |>
  left_join(vix_clean, by = "Date")

# 2) inner_join: only dates present in ALL datasets
inner_example <- assets_wide |>
  inner_join(cpi_clean, by = "Date") |>
  inner_join(dgs10_clean, by = "Date") |>
  inner_join(fedfunds_clean, by = "Date") |>
  inner_join(vix_clean, by = "Date")

# 3) full_join: all dates from any dataset
full_example <- assets_wide |>
  full_join(cpi_clean, by = "Date") |>
  full_join(dgs10_clean, by = "Date") |>
  full_join(fedfunds_clean, by = "Date") |>
  full_join(vix_clean, by = "Date")

# 4) anti_join: asset dates with NO macro data
anti_example <- assets_wide |>
  anti_join(cpi_clean, by = "Date")

# Build the final analytical dataset
final_dataset <- left_example

# Observation reduction table

reduction_table <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
    ↪ "VIX",
    "Assets (wide)", "Final Dataset"),
  Original_Obs = c(obs_original$Original_Obs, NA, NA),
  After_Cleaning = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
    nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix),
    nrow(assets_wide), NA),
  After_Joins = c(rep(NA, 8), NA, nrow(final_dataset))
)

```

Data Cleaning Summary

The following table presents a summary of the data dimensions before and after the cleaning and join operations, tracking the number of rows, columns, and missing values.

```
library(knitr)
cat(kable(reduction_table, format = "latex", booktabs = TRUE), file =
  ↪  "../reports/sections/data_cleaning_table.tex")
```

Sec 6: SQL-Based Financial Investigation – SISON, Aaron Joshua Estacio (Q1-Q3, Q8-Q10)

10 meaningful financial questions using select, filter, mutate, arrange, group_by, summarise.

```
# SQLite database setup for SQL-based investigations

con <- dbConnect(RSQLite::SQLite(), ":memory:")

# Build long-format analytical dataset for SQL queries
# Each row is one asset-date combination with macro variables attached
sql_long <- bind_rows(
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_AAPL, Daily_Return = Daily_Return_AAPL) |>
    mutate(Asset = "AAPL"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_BDO, Daily_Return = Daily_Return_BDO) |>
    mutate(Asset = "BDO"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_BTC, Daily_Return = Daily_Return_BTC) |>
    mutate(Asset = "BTC"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_SPY, Daily_Return = Daily_Return_SPY) |>
    mutate(Asset = "SPY")
)

# Pre-calculate CPI YoY for Q10 (inflation analysis)
# CPI is monthly; YoY change = (current / 12-month-ago) - 1
cpi_yoy <- cpi_clean |>
  arrange(Date) |>
  mutate(CPI_YoY = (CPI / lag(CPI, 12)) - 1) |>
```

```

select(Date, CPI_YoY)

# Join CPI_YoY into the long dataset and forward-fill to cover all trading
↪ days
sql_long <- sql_long |>
  left_join(cpi_yoy, by = "Date") |>
  arrange(Date) |>
  tidyr::fill(CPI_YoY, .direction = "down")

dbWriteTable(con, "integrated_data", sql_long, overwrite = TRUE)
fin_data <- tbl(con, "integrated_data")

```

Appendix A1: SQL Question 1

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To determine which asset delivered the highest return per unit of risk over the sample period. **Purpose of Code:** Calculates the ratio of mean daily return to standard deviation for each asset and visualizes the results. **Expected Output:** A risk-adjusted return ratio for each asset, presented as a bar chart.

```

# Q1: Risk-Adjusted Daily Return by Asset -- SISON, Aaron Joshua Estacio

q1 <- fin_data |>
  group_by(Asset) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE)
  ) |>
  mutate(Risk_Adjusted_Return = Mean_Daily_Return / SD_Daily_Return) |>
  arrange(desc(Risk_Adjusted_Return)) |>
  collect()

spy_ratio <- q1 |> filter(Asset == "SPY") |> pull(Risk_Adjusted_Return)

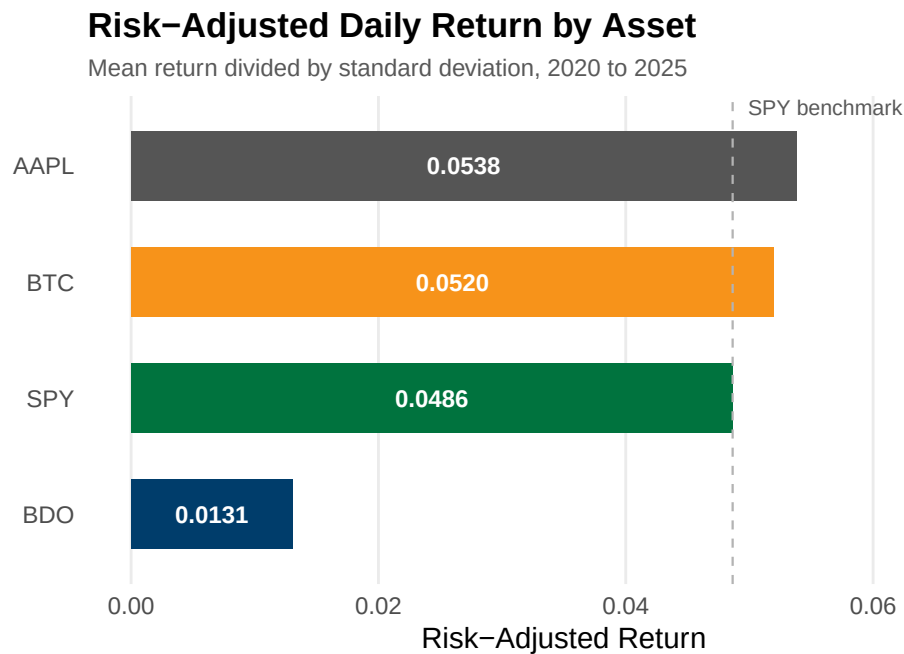
ggplot(q1, aes(x = Risk_Adjusted_Return, y = reorder(Asset,
↪ Risk_Adjusted_Return),
  fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = spy_ratio, linetype = "dashed",
    color = "#B3B3B3", linewidth = 0.4) +
  geom_text(aes(x = Risk_Adjusted_Return / 2,
    label = sprintf("%.4f", Risk_Adjusted_Return)),
    hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +

```

```

annotate("text", x = spy_ratio, y = 4.5, label = "SPY benchmark",
        size = 2.8, color = "#595959", hjust = -0.1) +
scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                             BTC = "#F7931A", SPY = "#00733E")) +
labs(title = "Risk-Adjusted Daily Return by Asset",
     subtitle = "Mean return divided by standard deviation, 2020 to 2025",
     x = "Risk-Adjusted Return", y = NULL) +
xlim(0, 0.07) +
theme_minimal(base_size = 11) +
theme(panel.grid.major.y = element_blank(),
      panel.grid.minor = element_blank(),
      axis.ticks = element_blank(),
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(size = 9, color = "#595959"))

```



```

ggsave("../reports/figures/fig_q1.pdf", width = 7, height = 3.5, device =
  "pdf")

```

Appendix A2: SQL Question 2

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To measure crisis resilience during the COVID-19 market crash. **Purpose of Code:** Computes the cumulative return of each asset over the February to March 2020 period. **Expected Output:** A line chart showing the trajectory of each asset's drawdown and recovery.

```

# Q2: COVID-19 Crash Cumulative Returns -- SISON, Aaron Joshua Estacio

q2_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q2 <- q2_raw |>
  filter(Date >= as.Date("2020-02-01") & Date <= as.Date("2020-03-31")) |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1)

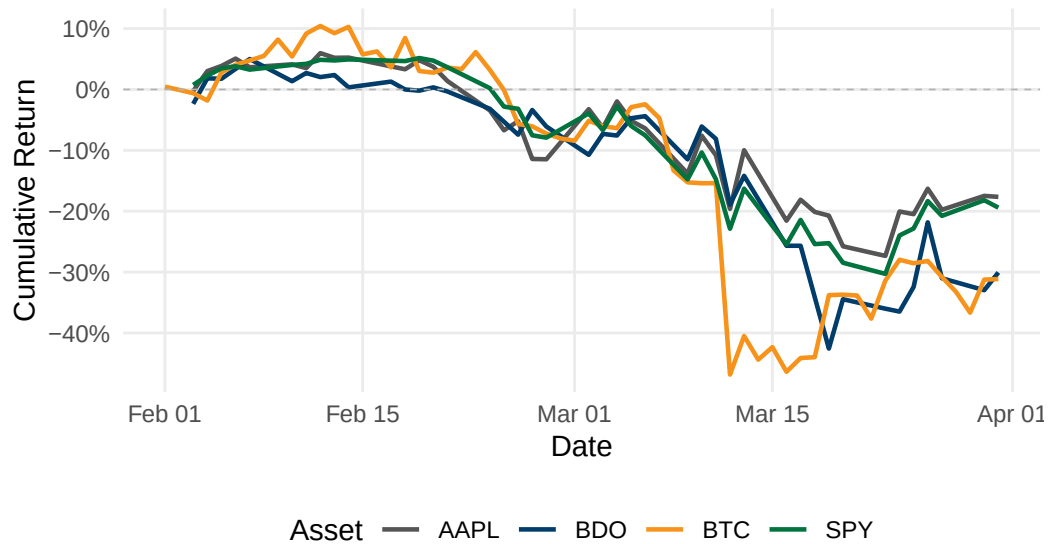
q2_summary <- q2 |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Total_Cumulative_Return = last(Cumulative_Return),
    Observations = n()
  )

ggplot(q2, aes(x = Date, y = Cumulative_Return, color = Asset)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#B3B3B3",
    ↪ linewidth = 0.3) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Cumulative Returns During the COVID-19 Crash",
    subtitle = "February to March 2020, indexed to 100% at period start",
    x = "Date", y = "Cumulative Return", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom")

```

Cumulative Returns During the COVID-19 Crash

February to March 2020, indexed to 100% at period start



```
ggsave("../reports/figures/fig_q2.pdf", width = 7, height = 4, device =
  "pdf")
```

Appendix A3: SQL Question 3

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To evaluate the cross-asset correlation of daily returns. **Purpose of Code:** Calculates and visualizes the pairwise Pearson correlation coefficients across the selected assets. **Expected Output:** A correlation heatmap matrix.

```
# Q3: Pairwise Correlation of Daily Returns -- SISON, Aaron Joshua Estacio

returns_wide <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  pivot_wider(names_from = Asset, values_from = Daily_Return)

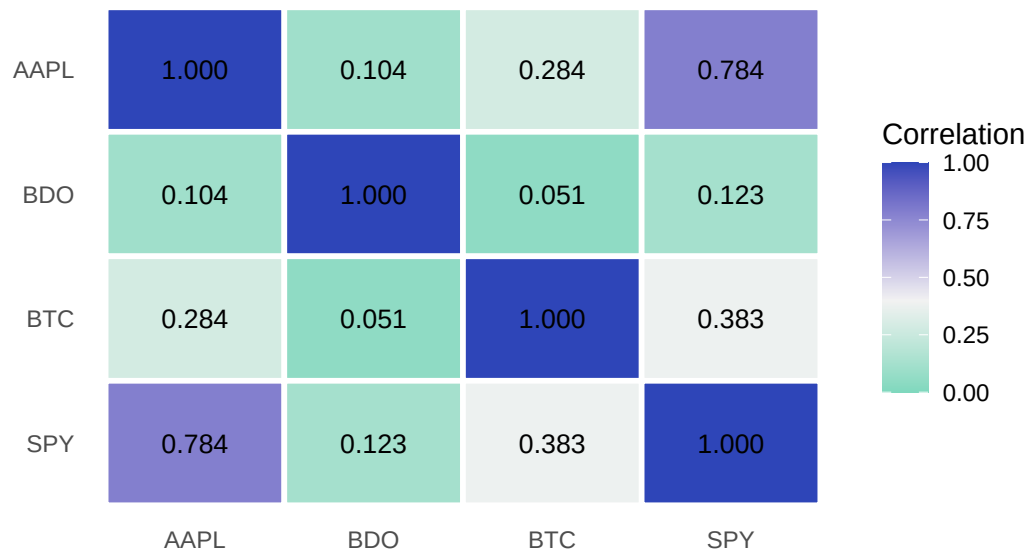
cor_matrix <- returns_wide |>
  select(AAPL, BDO, BTC, SPY) |>
  cor(use = "pairwise.complete.obs")

cor_long <- as.data.frame(cor_matrix) |>
  tibble::rownames_to_column("Asset1") |>
  tidyr::pivot_longer(-Asset1, names_to = "Asset2", values_to =
    "Correlation") |>
  mutate(Asset1 = factor(Asset1, levels = rev(c("AAPL", "BDO", "BTC",
    "SPY"))),
    Asset2 = factor(Asset2, levels = c("AAPL", "BDO", "BTC", "SPY")))
```

```
ggplot(cor_long, aes(x = Asset2, y = Asset1, fill = Correlation)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.3f", Correlation)), size = 3.5) +
  scale_fill_gradient2(low = "#1DC9A4", mid = "#F2F2F2", high = "#2E45B8",
    midpoint = 0.4, limits = c(0, 1)) +
  labs(title = "Pairwise Correlation of Daily Returns",
    subtitle = "Pearson correlation coefficients, 2020 to 2025",
    x = NULL, y = NULL, fill = "Correlation") +
  theme_minimal(base_size = 11) +
  theme(panel.grid = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "right")
```

Pairwise Correlation of Daily Returns

Pearson correlation coefficients, 2020 to 2025



```
ggsave("../reports/figures/fig_q3.pdf", width = 6, height = 4.5, device =
  ↪ "pdf")
```

Appendix A4: SQL Question 4

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To rank assets by their total price return from 2020 to 2025. **Purpose of Code:** Calculates the total and annualized returns for each asset across the entire sample period. **Expected Output:** A bar chart ranking the assets by total percentage return.

```

# Q4: Asset Ranking by Total Return -- CRUZ, Ricardo Miguel Inigo

q4_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q4 <- q4_raw |>
  filter(!is.na(Close), !is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Start_Close = first(Close),
    End_Close = last(Close),
    Total_Return = (End_Close / Start_Close) - 1,
    Annualized_Return = (1 + Total_Return)^(365.25 / as.numeric(End_Date -
      ↪ Start_Date)) - 1,
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(desc(Total_Return))

q4_plot <- q4 |>
  mutate(Asset = factor(Asset, levels = q4$Asset[order(q4$Total_Return)]))

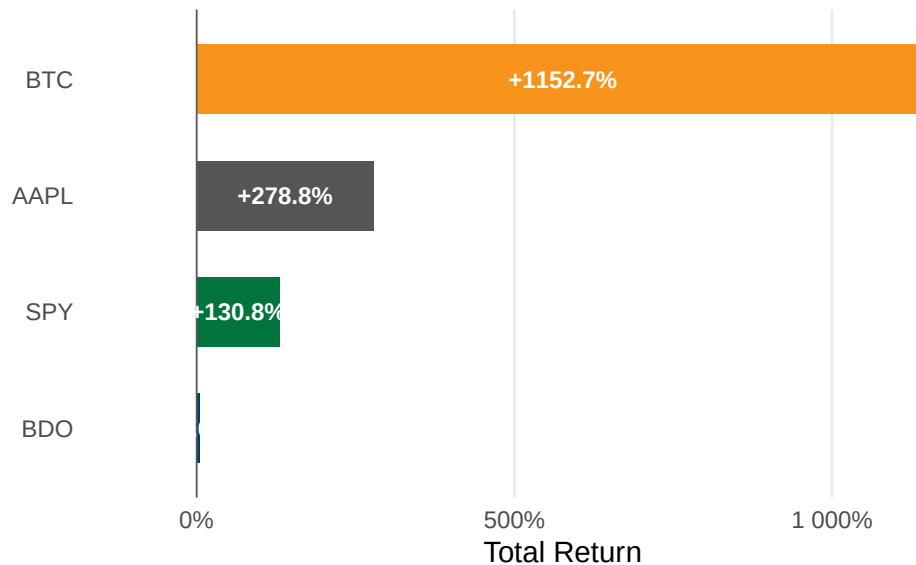
ggplot(q4_plot, aes(x = Total_Return, y = Asset, fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = 0, linewidth = 0.3, color = "#595959") +
  geom_text(aes(x = Total_Return / 2,
    label = sprintf("%+.1f%", Total_Return * 100)),
    hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
    BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format(),
    expand = expansion(mult = c(0.15, 0.15))) +
  labs(title = "Asset Ranking by Total Return",
    subtitle = "Total price return from 2020 to 2025",
    x = "Total Return", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),

```

```
axis.ticks = element_blank(),
plot.title = element_text(face = "bold", size = 13),
plot.subtitle = element_text(size = 9, color = "#595959"))
```

Asset Ranking by Total Return

Total price return from 2020 to 2025



```
ggsave("../reports/figures/fig_q4.pdf", width = 7, height = 3.5, device =
  "pdf")
```

Appendix A5: SQL Question 5

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To identify the extreme single-day gains and losses for each asset. **Purpose of**

Code: Extracts the maximum and minimum daily returns for each instrument to assess tail risk. **Expected Output:** A grouped bar chart displaying the best and worst trading days.

```
# Q5: Best and Worst Trading Days -- CRUZ, Ricardo Miguel Inigo
```

```
q5_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q5 <- q5_raw |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  summarise(
    Best_Date = Date[which.max(Daily_Return)],
    Best_Daily_Return = max(Daily_Return, na.rm = TRUE),
```

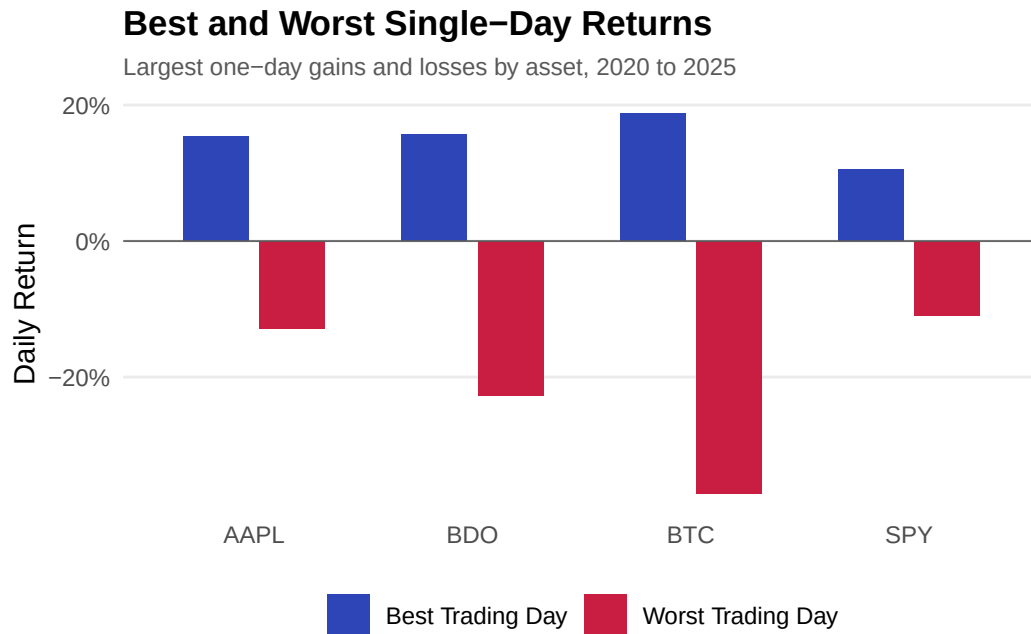
```

    Worst_Date = Date[which.min(Daily_Return)],
    Worst_Daily_Return = min(Daily_Return, na.rm = TRUE),
    Return_Range = Best_Daily_Return - Worst_Daily_Return,
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Worst_Daily_Return)

q5_plot <- q5 |>
  select(Asset, Best_Daily_Return, Worst_Daily_Return) |>
  pivot_longer(cols = c(Best_Daily_Return, Worst_Daily_Return),
               names_to = "Trading_Day_Type",
               values_to = "Daily_Return") |>
  mutate(
    Trading_Day_Type = recode(Trading_Day_Type,
                              Best_Daily_Return = "Best Trading Day",
                              Worst_Daily_Return = "Worst Trading Day")
  )

ggplot(q5_plot, aes(x = Asset, y = Daily_Return, fill = Trading_Day_Type)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("Best Trading Day" = "#2E45B8",
                              "Worst Trading Day" = "#C91D42")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Best and Worst Single-Day Returns",
       subtitle = "Largest one-day gains and losses by asset, 2020 to 2025",
       x = NULL, y = "Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")

```



```
ggsave("../reports/figures/fig_q5.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Appendix A6: SQL Question 6

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To analyze momentum regime shifts using moving averages. **Purpose of Code:** Implements a 20-day and 60-day simple moving average crossover strategy. **Expected Output:** A multi-panel line chart showing crossover events and momentum trends.

```
# Q6: Moving Average Crossover Analysis -- CRUZ, Ricardo Miguel Inigo

q6_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q6_ma <- q6_raw |>
  filter(!is.na(Close)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(
    MA_20 = as.numeric(stats::filter(Close, rep(1 / 20, 20), sides = 1)),
    MA_60 = as.numeric(stats::filter(Close, rep(1 / 60, 60), sides = 1)),
    Signal = case_when(
      MA_20 > MA_60 ~ "Bullish",
      MA_20 < MA_60 ~ "Bearish",
      TRUE ~ "Neutral"
```

```

    )
  ) |>
  ungroup()

q6 <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60), !is.na(Daily_Return)) |>
  group_by(Asset, Signal) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, desc(Mean_Daily_Return))

q6_latest_signal <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60)) |>
  group_by(Asset) |>
  slice_max(Date, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(Asset, Date, Close, MA_20, MA_60, Signal) |>
  arrange(Asset)

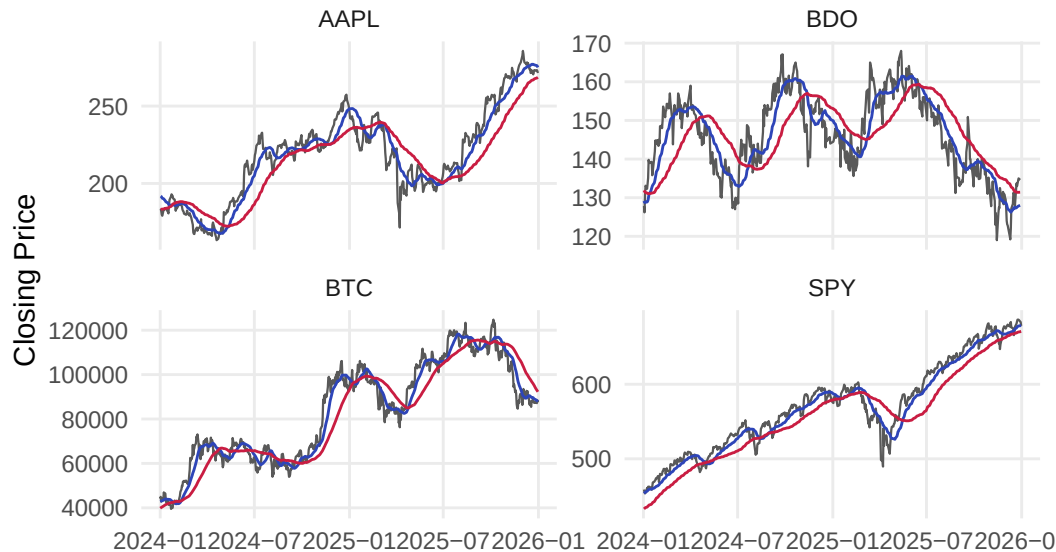
q6_plot <- q6_ma |>
  filter(Date >= as.Date("2024-01-01"))

ggplot(q6_plot, aes(x = Date)) +
  geom_line(aes(y = Close), linewidth = 0.4, color = "#595959") +
  geom_line(aes(y = MA_20), linewidth = 0.5, color = "#2E45B8") +
  geom_line(aes(y = MA_60), linewidth = 0.5, color = "#C91D42") +
  facet_wrap(~ Asset, scales = "free_y") +
  labs(title = "20-Day and 60-Day Moving Average Crossover",
       subtitle = "Bullish signal when 20-day MA is above 60-day MA, 2024 to
       ↪ 2025",
       x = NULL, y = "Closing Price") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))

```

20-Day and 60-Day Moving Average Crossover

Bullish signal when 20-day MA is above 60-day MA, 2024 to 2025



```
ggsave("../reports/figures/fig_q6.pdf", width = 7, height = 4.5, device =
  ↪ "pdf")
```

Appendix A7: SQL Question 7

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To test whether elevated trading volume systematically coincides with higher volatility. **Purpose of Code:** Partitions trading days into high-volume and normal-volume regimes and compares their mean absolute returns. **Expected Output:** A grouped bar chart comparing volatility across volume regimes.

```
# Q7: Volume and Volatility Relationship -- CRUZ, Ricardo Miguel Inigo
```

```
volume_lookup <- assets_all |>
  select(Date, Asset, Volume) |>
  mutate(
    Date = as.Date(Date),
    Volume = as.numeric(Volume)
  )

q7_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(
    Date = as.Date(Date),
    Daily_Return = as.numeric(Daily_Return)
  ) |>
```

```

left_join(volume_lookup, by = c("Date", "Asset")) |>
filter(
  !is.na(Daily_Return),
  !is.na(Volume),
  Volume > 0
)

q7 <- q7_raw |>
group_by(Asset) |>
mutate(
  Volume_Threshold = quantile(Volume, 0.75, na.rm = TRUE),
  Volume_Regime = if_else(
    Volume >= Volume_Threshold,
    "High Volume Days",
    "Normal Volume Days"
  ),
  Absolute_Return = abs(Daily_Return)
) |>
ungroup() |>
group_by(Asset, Volume_Regime) |>
summarise(
  Observations = n(),
  Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
  Mean_Absolute_Return = mean(Absolute_Return, na.rm = TRUE),
  Volatility = sd(Daily_Return, na.rm = TRUE),
  Average_Volume = mean(Volume, na.rm = TRUE),
  .groups = "drop"
) |>
arrange(Asset, Volume_Regime)

q7

```

```

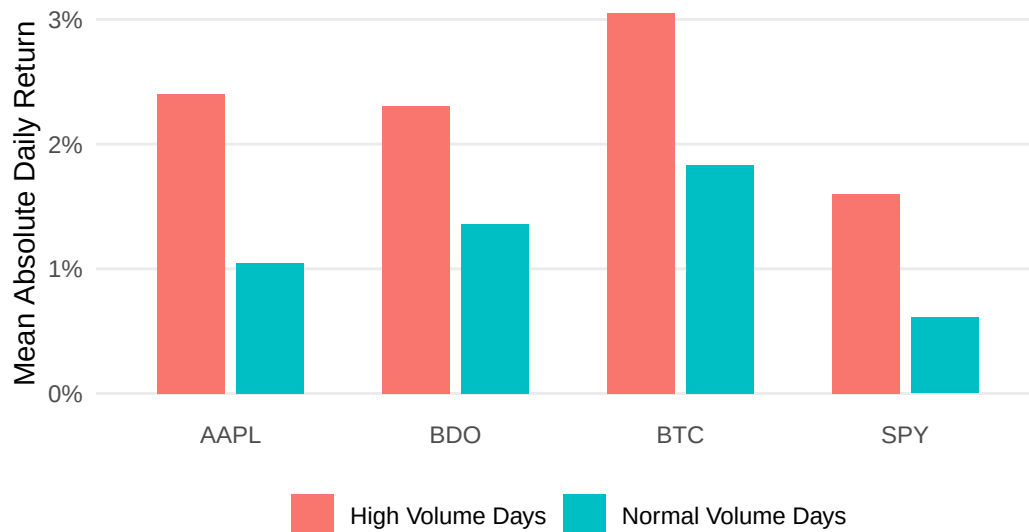
# A tibble: 8 x 7
  Asset Volume_Regime Observations Mean_Daily_Return
  <chr> <chr>          <int>          <dbl>
1 AAPL High Volume Days      377          0.00155
2 AAPL Normal Volume Days    1130          0.000921
3 BDO  High Volume Days      367         -0.0000981
4 BDO  Normal Volume Days    1100          0.000426
5 BTC  High Volume Days      548          0.00168
6 BTC  Normal Volume Days    1643          0.00165
7 SPY  High Volume Days      377         -0.00292
8 SPY  Normal Volume Days    1130          0.00182
# i 3 more variables: Mean_Absolute_Return <dbl>, Volatility <dbl>,
#   Average_Volume <dbl>

```

```
ggplot(q7, aes(x = Asset, y = Mean_Absolute_Return, fill = Volume_Regime)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Volume and Volatility Relationship",
    subtitle = "Mean absolute daily return during high-volume versus
      ↪ normal-volume days, 2020 to 2025",
    x = NULL,
    y = "Mean Absolute Daily Return",
    fill = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom"
  )
)
```

Volume and Volatility Relationship

Mean absolute daily return during high-volume versus normal-volume days, 2020 to 2025



```
dir.create("../reports/figures", recursive = TRUE, showWarnings = FALSE)

ggsave("../reports/figures/fig_q7.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Appendix A8: SQL Question 8

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To assess asset performance during periods of high versus low market fear.

Purpose of Code: Segregates trading days based on VIX levels and calculates conditional mean daily returns. **Expected Output:** A grouped bar chart comparing returns under different VIX regimes.

```
# Q8: VIX Regime Analysis -- SISON, Aaron Joshua Estacio

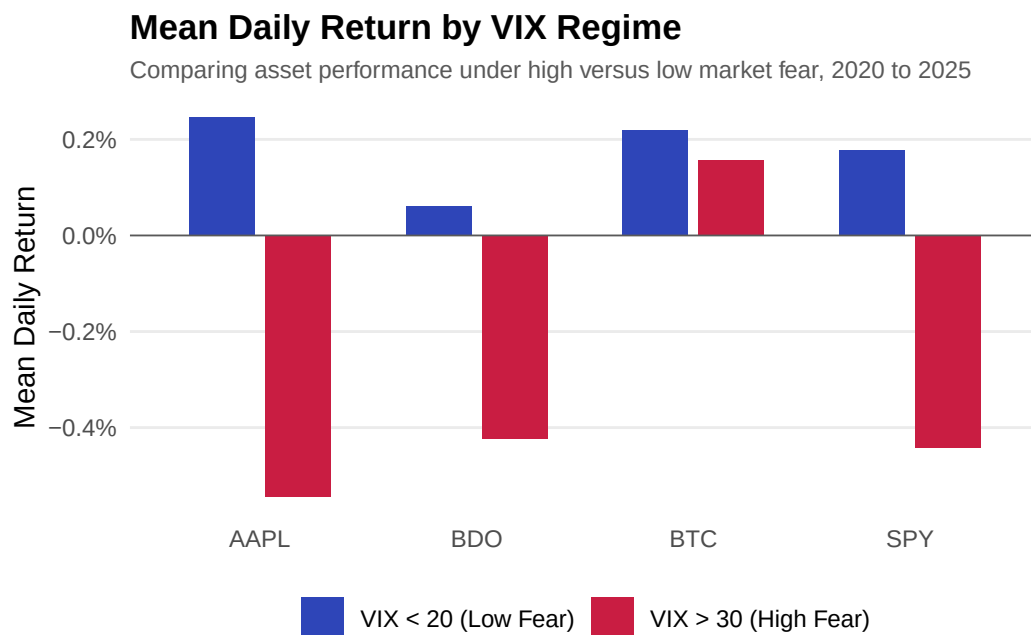
q8_raw <- fin_data |>
  filter(!is.na(VIX)) |>
  select(Asset, Date, Daily_Return, VIX) |>
  collect() |>
  mutate(Date = as.Date(Date))

q8 <- q8_raw |>
  mutate(
    VIX_Regime = case_when(
      VIX > 30 ~ "High Volatility (VIX > 30)",
      VIX < 20 ~ "Low Volatility (VIX < 20)",
      TRUE ~ "Moderate (20 <= VIX <= 30)"
    )
  ) |>
  group_by(Asset, VIX_Regime) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, VIX_Regime)

q8_plot <- q8 |>
  filter(VIX_Regime %in% c("High Volatility (VIX > 30)", "Low Volatility
    ↵ (VIX < 20)")) |>
  mutate(VIX_Regime = recode(VIX_Regime,
    "High Volatility (VIX > 30)" = "VIX > 30 (High Fear)",
    "Low Volatility (VIX < 20)" = "VIX < 20 (Low Fear)"))

ggplot(q8_plot, aes(x = Asset, y = Mean_Daily_Return, fill = VIX_Regime)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("VIX > 30 (High Fear)" = "#C91D42",
    "VIX < 20 (Low Fear)" = "#2E45B8")) +
```

```
scale_y_continuous(labels = scales::percent_format()) +
labs(title = "Mean Daily Return by VIX Regime",
      subtitle = "Comparing asset performance under high versus low market
      ↪ fear, 2020 to 2025",
      x = NULL, y = "Mean Daily Return", fill = NULL) +
theme_minimal(base_size = 11) +
theme(panel.grid.major.x = element_blank(),
      panel.grid.minor = element_blank(),
      axis.ticks = element_blank(),
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(size = 9, color = "#595959"),
      legend.position = "bottom")
```



```
ggsave("../reports/figures/fig_q8.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Appendix A9: SQL Question 9

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To measure asset sensitivity to changes in the 10-year Treasury yield. **Purpose of Code:** Calculates mean returns conditional on daily increases or decreases in the DGS10 rate. **Expected Output:** A grouped bar chart showing returns conditioned on yield direction.

```
# Q9: Interest Rate Sensitivity -- SISON, Aaron Joshua Estacio

q9_raw <- fin_data |>
```

```

filter(!is.na(DGS10)) |>
select(Asset, Date, Daily_Return, DGS10) |>
collect() |>
mutate(Date = as.Date(Date))

q9 <- q9_raw |>
group_by(Asset) |>
arrange(Date) |>
mutate(DGS10_Change = DGS10 - lag(DGS10)) |>
ungroup() |>
filter(!is.na(DGS10_Change)) |>
mutate(
  Yield_Direction = case_when(
    DGS10_Change > 0 ~ "Yields Rising",
    DGS10_Change < 0 ~ "Yields Falling",
    TRUE ~ "No Change"
  )
) |>
group_by(Asset, Yield_Direction) |>
summarise(
  Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
  SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
  Observations = n(),
  .groups = "drop"
) |>
arrange(Asset, Yield_Direction)

q9_plot <- q9 |>
filter(Yield_Direction %in% c("Yields Rising", "Yields Falling"))

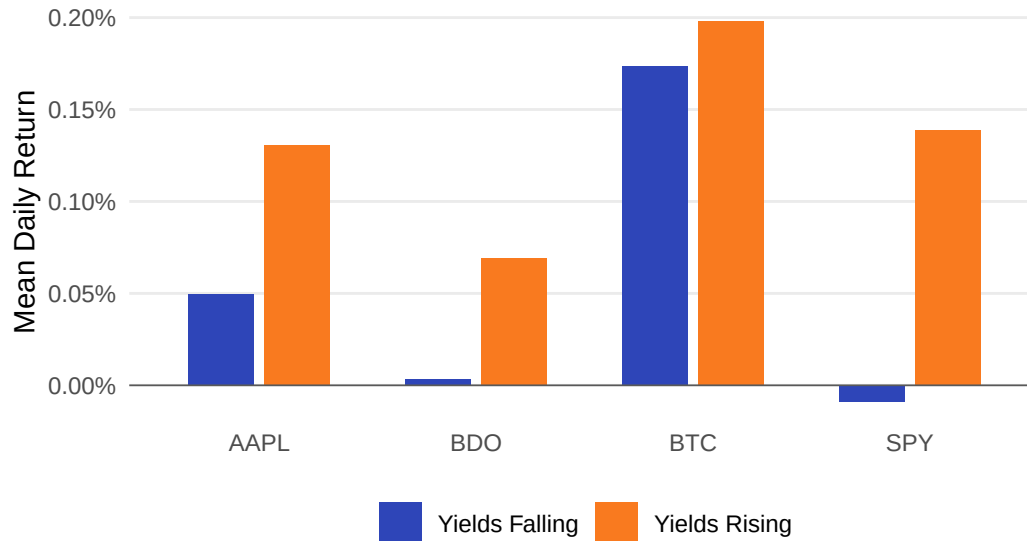
ggplot(q9_plot, aes(x = Asset, y = Mean_Daily_Return, fill =
  ↪ Yield_Direction)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("Yields Rising" = "#F97A1F",
    "Yields Falling" = "#2E45B8")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Mean Daily Return by 10-Year Treasury Yield Direction",
    subtitle = "Asset returns conditional on daily yield changes, 2020 to
    ↪ 2025",
    x = NULL, y = "Mean Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),

```

```
plot.title = element_text(face = "bold", size = 13),
plot.subtitle = element_text(size = 9, color = "#595959"),
legend.position = "bottom")
```

Mean Daily Return by 10-Year Treasury Yield Direction

Asset returns conditional on daily yield changes, 2020 to 2025



```
ggsave("../reports/figures/fig_q9.pdf", width = 7, height = 4, device =
  ↪ "pdf")
```

Appendix A10: SQL Question 10

Pertains to Section 6: SQL-Based Financial Investigation

Objective: To evaluate which assets served as effective inflation hedges from 2021 to 2023.

Purpose of Code: Computes total cumulative returns during the high-inflation window where CPI YoY exceeded 5%. **Expected Output:** A horizontal bar chart of cumulative returns during the inflationary period.

```
# Q10: Inflation Hedge Performance -- SISON, Aaron Joshua Estacio
```

```
q10_raw <- fin_data |>
  select(Date, Asset, Daily_Return, CPI_YoY) |>
  collect() |>
  mutate(Date = as.Date(Date))
```

```
q10 <- q10_raw |>
  filter(!is.na(CPI_YoY) & CPI_YoY > 0.05) |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
```

```

mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1) |>
summarise(
  Start_Date = min(Date),
  End_Date = max(Date),
  Start_CPI_YoY = first(CPI_YoY),
  End_CPI_YoY = last(CPI_YoY),
  Total_Cumulative_Return = last(Cumulative_Return),
  Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
  Observations = n()
) |>
arrange(desc(Total_Cumulative_Return))

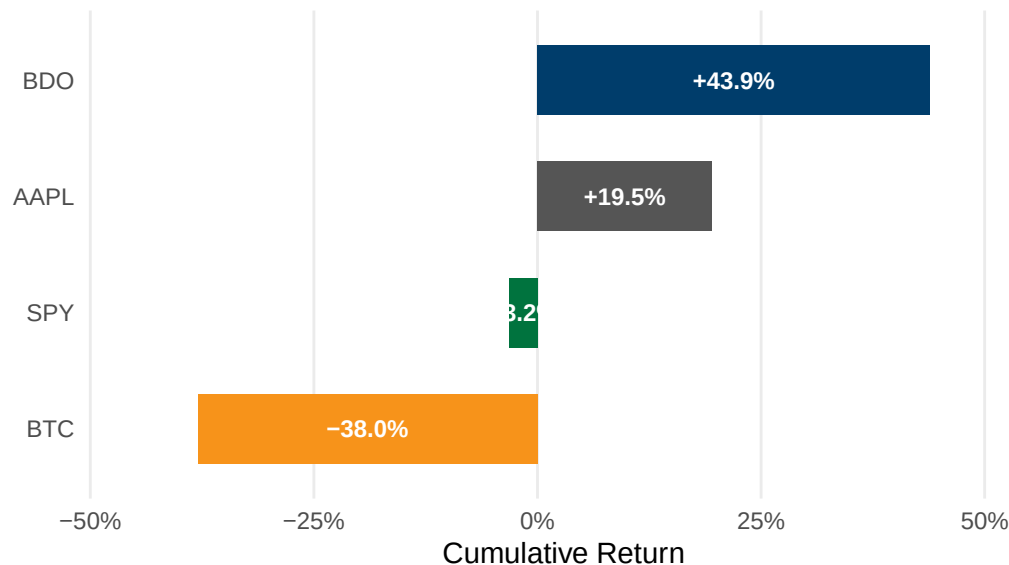
q10_plot <- q10 |>
mutate(Asset = factor(Asset, levels =
  ↪ q10$Asset[order(q10$Total_Cumulative_Return)]))

ggplot(q10_plot, aes(x = Total_Cumulative_Return, y = Asset, fill = Asset))
↪ +
geom_col(width = 0.6, show.legend = FALSE) +
geom_text(aes(x = Total_Cumulative_Return / 2,
  label = sprintf("%.1f%", Total_Cumulative_Return * 100)),
  hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
  BTC = "#F7931A", SPY = "#00733E")) +
scale_x_continuous(labels = scales::percent_format(),
  expand = expansion(mult = c(0.15, 0.15))) +
labs(title = "Cumulative Return During High Inflation Period",
  subtitle = "June 2021 to February 2023, CPI year-on-year above 5%",
  x = "Cumulative Return", y = NULL) +
theme_minimal(base_size = 11) +
theme(panel.grid.major.y = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_blank(),
  plot.title = element_text(face = "bold", size = 13),
  plot.subtitle = element_text(size = 9, color = "#595959"))

```

Cumulative Return During High Inflation Period

June 2021 to February 2023, CPI year-on-year above 5%



```
ggsave("../reports/figures/fig_q10.pdf", width = 7, height = 3.5, device =  
  ↪ "pdf")
```

Sec 7: Descriptive Statistics – GALEDO, Enrique Lorenzo Hermoso

Mean, median, variance, std dev, min, max, range, skewness, kurtosis.

```
# Sec 7: Descriptive Statistics -- compute mean, median, variance, skewness,  
  ↪ kurtosis -- GALEDO, Enrique Lorenzo Hermoso
```

Sec 8: Data Visualization – SEBALLOS, Josiah Dweyn Panganiban

At least 8 professional-quality figures.

```
# Sec 8: Data Visualization -- create 8 professional figures with ggplot2 --  
  ↪ SEBALLOS, Josiah Dweyn Panganiban
```

Sec 9: Exploratory Analysis – SEECHUNG, Camille Castro

Compare returns, volatility, inflationary periods, high-VIX periods, interest-rate environments.

```
# Sec 9: Exploratory Financial Analysis -- compare returns, volatility,  
  ↪ inflation, VIX, interest rates -- SEECHUNG, Camille Castro
```

Sec 10: Investment Recommendation – SEECHUNG, Camille Castro

Allocate USD 1,000,000.

```
# Sec 10: Investment Recommendation -- USD 1,000,000 allocation among AAPL,  
↪ BDO, BTC, cash -- SEECHUNG, Camille Castro
```

Sec 11: Executive Dashboard – CRUZ, Ricardo Miguel Iñigo

```
# Sec 11: Executive Dashboard -- key stats, charts, SQL findings,  
↪ recommendation summary -- CRUZ, Ricardo Miguel Iñigo
```